

Experiment 05 – Advanced SQL Queries

Aim

To implement and analyze advanced SQL queries using aggregate functions, conditional expressions, joins, grouping, and subqueries to solve real-world database problems.

Objective

- To calculate confirmation rates using SQL aggregate functions.
- To handle users with no confirmation requests.
- To calculate cancellation rates for valid trips.
- To apply joins and filtering conditions on multiple tables.
- To use conditional aggregation and rounding in SQL queries.

Question 1

A website maintains records of users who have registered on the platform and the status of confirmation requests sent to them. A confirmation request can either be successfully completed or may expire without completion.

For each registered user, calculate the confirmation rate, which is defined as the number of confirmed requests divided by the total number of confirmation requests.

If a user has never received any confirmation request, their confirmation rate should be considered 0.

Display the user ID along with the confirmation rate rounded to 2 decimal places. The result can be returned in any order.

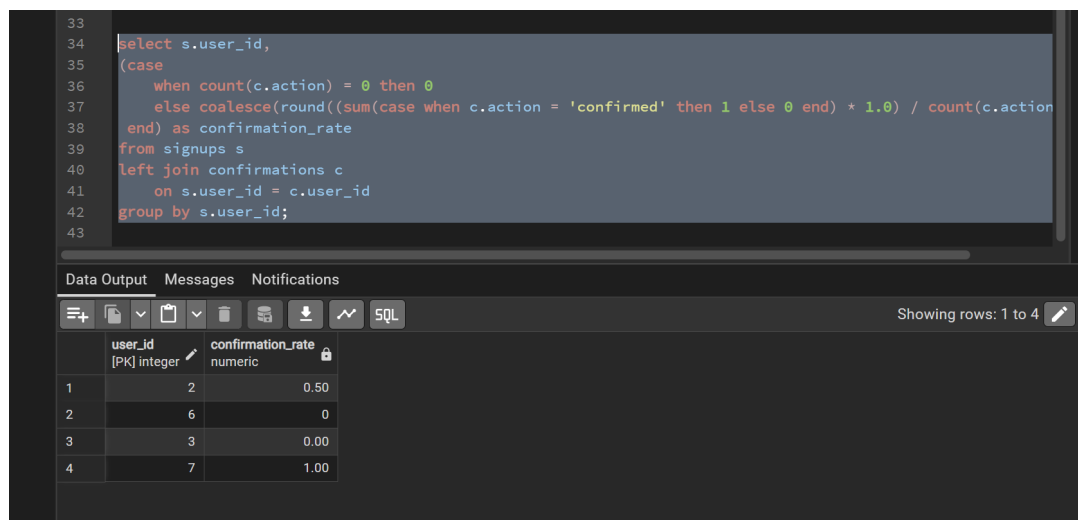
Answer

```

1 select s.user_id,
2 (case
3 when count(c.action) = 0 then 0
4 else coalesce(round((sum(case when c.action = 'confirmed' then 1 else 0 end) * 1.0)
5 / count(c.action)
6 end) as confirmation_rate
7 from signups s
8 Left join confirmations c
9 on s.user_id = c.user_id
group by s.user_id;

```

Output



The screenshot shows a SQL IDE interface. The top pane displays the SQL query. The bottom pane shows the 'Data Output' tab with the results of the query. The output table has two columns: 'user_id' (integer, primary key) and 'confirmation_rate' (numeric). The results show four rows of data.

	user_id [PK] integer	confirmation_rate numeric
1	2	0.50
2	6	0
3	3	0.00
4	7	1.00

Question 2

A ride-booking company maintains information about trip requests and the users involved in those trips. Some users may be restricted from using the platform, while others are active participants. Each trip can either be completed successfully or cancelled by one of the participants.

For each day between 2013-10-01 and 2013-10-03, determine the proportion of canceled trips among all valid trips.

A trip should be considered valid only when both the customer and the driver associated with that trip are active users.

The final result should display the date and the calculated cancellation rate rounded to two decimal places.

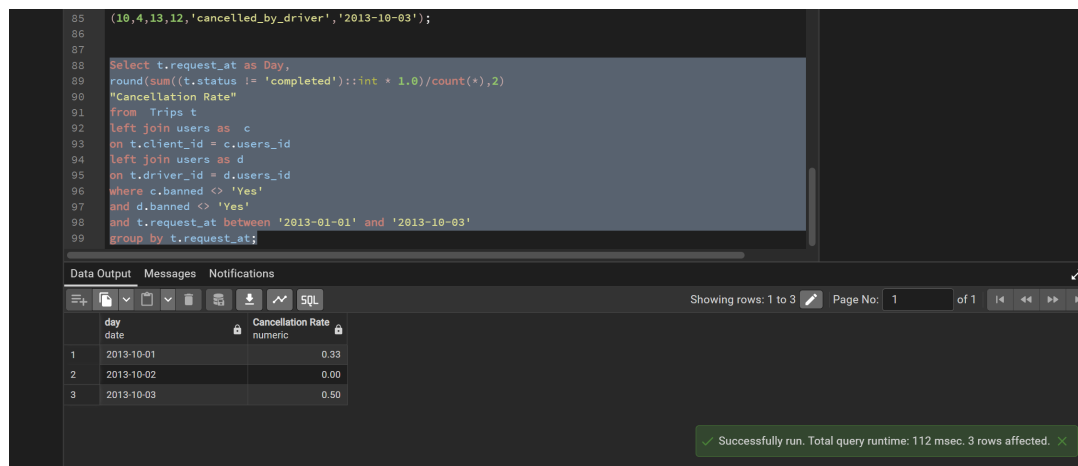
Answer

```

1 Select t.request_at as Day,
2 round(sum((t.status != 'completed') :: int * 1.0)/count(*),2)
3 "Cancellation Rate"
4 from Trips t
5 Left join users as c
6 on t.client_id = c.users_id
7 left join users as d
8 on t.driver_id = d.users_id
9 where c.banned <> 'Yes'
10 and d.banned <> 'Yes'
11 and t.request_at between '2013-01-01' and '2013-10-03'
12 group by t.request_at;

```

Output



The screenshot shows a SQL query execution interface. The query is the same as the one in the 'Answer' section. The output is displayed in a table with two columns: 'day' and 'Cancellation Rate'. The results are for the dates 2013-10-01, 2013-10-02, and 2013-10-03.

day	Cancellation Rate
2013-10-01	0.33
2013-10-02	0.00
2013-10-03	0.50

Below the table, a status message indicates: "Successfully run. Total query runtime: 112 msec. 3 rows affected."

Learning Outcome

After completing this experiment, the learner will be able to:

- Calculate rates and percentages using SQL aggregate functions.
- Use CASE expressions for conditional aggregation.
- Apply LEFT JOIN to retain users who do not have corresponding confirmation records.
- Use multiple joins to validate records across related tables.
- Filter records using multiple conditions and date ranges.
- Use ROUND() and COALESCE() to produce appropriate formatted results.